

บทที่ 2

แนะนำเครื่องมือพัฒนาโปรแกรม

ในบทนี้เราจะสำรวจเครื่องมือที่จำเป็นสำหรับการพัฒนาโปรแกรมด้วยภาษาไพธอน ตั้งแต่การติดตั้งตัวแปลภาษา (Interpreter) ไปจนถึงการเลือกใช้โปรแกรมแก้ไขโค้ด (Code Editor) และสภาพแวดล้อมการพัฒนาแบบผสมผสาน (Integrated Development Environment - IDE) โดยพื้นฐานแล้ว สคริปต์ไพธอน (Python Script) คือไฟล์ข้อความธรรมดา (Plain Text) ที่มีนามสกุล `.py` ซึ่งหมายความว่าเราสามารถเริ่มต้นเขียนโปรแกรมไพธอนได้ด้วยโปรแกรมแก้ไขข้อความทั่วไปและตัวแปลภาษาไพธอนที่ติดตั้งในเครื่องคอมพิวเตอร์

อย่างไรก็ตาม เพื่อเพิ่มประสิทธิภาพและความสะดวกในการพัฒนาโปรแกรม นักพัฒนาส่วนใหญ่นิยมใช้เครื่องมือที่มีคุณสมบัติสูงขึ้น เช่น โปรแกรมแก้ไขโค้ดขั้นสูงอย่าง Visual Studio Code¹, Sublime Text² หรือ IDE ที่ออกแบบมาสำหรับภาษาไพธอนโดยเฉพาะ เช่น PyCharm³ และ Spyder⁴

สำหรับงานด้านวิทยาศาสตร์ข้อมูล (Data Science) และการเรียนรู้ของเครื่อง (Machine Learning) เครื่องมือที่ได้รับความนิยมอย่างสูงคือ Jupyter Notebook และ JupyterLab ซึ่งเป็นเว็บแอปพลิเคชันที่ช่วยให้สามารถเขียนโค้ด, สร้างเอกสาร, แสดงผลลัพธ์ และสร้างภาพกราฟิกได้ในที่เดียว ทำให้เหมาะกับการทดลองและนำเสนอผลการวิเคราะห์ข้อมูล

2.1 การติดตั้งตัวแปลภาษาไพธอน

ในส่วนนี้จะแนะนำวิธีการติดตั้งตัวแปลภาษาไพธอนเวอร์ชันล่าสุด (ณ เดือนกรกฎาคม 2568 คือเวอร์ชัน 3.13) บนระบบปฏิบัติการหลัก ได้แก่ Microsoft Windows 11, macOS Sequoia 15 และ Ubuntu Linux 24.04 LTS

2.1.1 การติดตั้งบนระบบปฏิบัติการ Microsoft Windows 11

สำหรับผู้ใช้งาน Microsoft Windows 11 วิธีที่ง่ายและแนะนำคือการติดตั้งผ่าน Microsoft Store ซึ่งจะช่วยจัดการการตั้งค่า PATH ให้โดยอัตโนมัติ

1. เปิดโปรแกรม Microsoft Store และค้นหาด้วยคำว่า "Python"
2. เลือกเวอร์ชันล่าสุดที่ปรากฏ (เช่น Python 3.13) และคลิก 'Get' หรือ 'Install' เพื่อเริ่มกระบวนการติดตั้ง
3. เมื่อติดตั้งเสร็จสิ้น ให้เปิดโปรแกรม Command Prompt หรือ PowerShell ขึ้นมา แล้วพิมพ์คำสั่งต่อไปนี้เพื่อตรวจสอบเวอร์ชัน

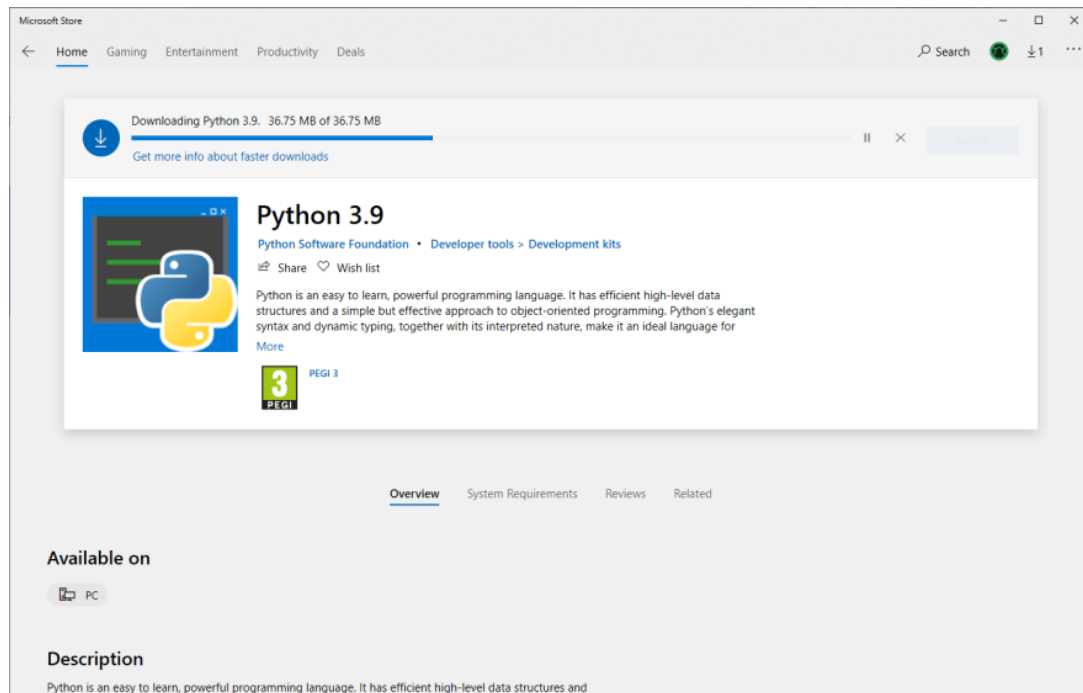
```
python --version  
pip --version
```

¹Microsoft Visual Studio Code, ดูเพิ่มเติมได้ที่ <https://code.visualstudio.com>

²Sublime Text: Text Editing, Done Right, ดูเพิ่มเติมได้ที่ <https://www.sublimetext.com>

³PyCharm: The Python IDE for Professional Developers, ดูเพิ่มเติมได้ที่ <https://www.jetbrains.com/pycharm/>

⁴Spyder: The Scientific Python Development Environment, ดูเพิ่มเติมได้ที่ <https://www.spyder-ide.org>



รูป 2.1: การติดตั้งตัวแปลภาษาไพธอนผ่าน Microsoft Store (Installation of Python via Microsoft Store)

ผลลัพธ์ที่แสดงควรเป็นเวอร์ชันของ Python และ pip ที่คุณเพิ่งติดตั้งไป

2.1.2 การติดตั้งบนระบบปฏิบัติการ Ubuntu Linux 24.04 LTS

ระบบปฏิบัติการตระกูล Ubuntu Linux 24.04 LTS (เช่น Ubuntu) มักจะมี Python 3 ติดตั้งมาแล้ว อย่างไรก็ตาม เราสามารถอัปเดตและติดตั้งเครื่องมือที่จำเป็นเพิ่มเติมได้ดังนี้

1. เปิดโปรแกรม Terminal และใช้คำสั่งต่อไปนี้เพื่ออัปเดตระบบ

```
sudo apt update
sudo apt -y upgrade
```

2. ตรวจสอบเวอร์ชันของ Python ที่มีอยู่ด้วยคำสั่ง

```
python3 --version
```

3. ติดตั้ง 'pip' ซึ่งเป็นโปรแกรมสำหรับจัดการแพ็คเกจของไพธอน

```
sudo apt install -y python3-pip
```

4. ตรวจสอบเวอร์ชันของ 'pip' เพื่อยืนยันการติดตั้ง

```
pip3 --version
```

2.1.3 การติดตั้งบนระบบปฏิบัติการ macOS Sequoia 15

สำหรับ macOS Sequoia 15 แนะนำให้ติดตั้งผ่าน Homebrew ซึ่งเป็นโปรแกรมจัดการแพ็คเกจยอดนิยม

1. หากยังไม่มี Homebrew ให้ติดตั้งโดยเปิด Terminal และวางคำสั่งจากเว็บไซต์ทางการ <https://brew.sh>
2. อัปเดต Homebrew และติดตั้ง Python เวอร์ชันล่าสุด

```
brew update  
brew install python
```

3. หลังจากติดตั้งเสร็จสิ้น ให้ตรวจสอบเวอร์ชันของ Python และ pip

```
python3 --version  
pip3 --version
```

2.2 สภาพแวดล้อมเสมือน (Virtual Environments)

ในการพัฒนาโปรแกรมจริง เรามักจะทำงานกับหลายโปรเจกต์ ซึ่งแต่ละโปรเจกต์อาจต้องการไลบรารี (Library) หรือแพ็คเกจ (Package) ในเวอร์ชันที่แตกต่างกัน การใช้สภาพแวดล้อมเสมือน หรือ Virtual Environment จะช่วยแยกโปรเจกต์ออกจากกัน ทำให้การจัดการเวอร์ชันของแพ็คเกจไม่ขัดแย้งกัน โมดูล venv เป็นเครื่องมือมาตรฐานที่มาพร้อมกับไพธอน

1. การสร้างสภาพแวดล้อมเสมือน: เปิด Terminal หรือ Command Prompt ไปยังโฟลเดอร์โปรเจกต์ของคุณ แล้วใช้คำสั่ง

```
python -m venv myenv
```

คำสั่งนี้จะสร้างโฟลเดอร์ชื่อ myenv ซึ่งบรรจุไฟล์ของสภาพแวดล้อมเสมือน

2. การเปิดใช้งาน (Activate):

- บน Microsoft Windows 11:

```
myenv\Scripts\activate
```

- บน macOS Sequoia 15 และ Ubuntu Linux 24.04 LTS:

```
source myenv/bin/activate
```

เมื่อเปิดใช้งานแล้ว ชื่อของสภาพแวดล้อมเสมือนจะปรากฏขึ้นที่หน้าพrompt ((myenv) ...\$)

3. การปิดใช้งาน (Deactivate): เพียงพิมพ์คำสั่ง deactivate

การใช้สภาพแวดล้อมเสมือนเป็นแนวปฏิบัติที่ดี (Best Practice) ที่นักพัฒนาไพธอนทุกคนควรทำความคุ้นเคย

2.3 การใช้งานไพธอนสคริปต์เบื้องต้น

หลังจากติดตั้งตัวแปลภาษาเรียบร้อยแล้ว เรามาลองสร้างและรันสคริปต์ไพธอนอย่างง่ายกัน

1. เปิดโปรแกรม Text Editor ที่คุณถนัด (เช่น Notepad, VS Code) แล้วสร้างไฟล์ใหม่
2. พิมพ์โค้ดต่อไปนี้ลงในไฟล์

```
# My first Python script
print('Hello, Modern Python!')
```

3. บันทึกไฟล์โดยตั้งชื่อให้ลงท้ายด้วยนามสกุล .py เช่น hello.py
4. เปิด Terminal หรือ Command Prompt แล้วใช้คำสั่ง cd เพื่อเข้าไปยังตำแหน่งที่บันทึกไฟล์ไว้ จากนั้นรันสคริปต์ด้วยคำสั่ง

```
python hello.py
```

(หรือใช้ python3 ขึ้นอยู่กับระบบ) ผลลัพธ์คือข้อความ Hello, Modern Python! จะถูกแสดงบนหน้าจอ

2.4 รู้จักกับ Jupyter Notebook

Jupyter Notebook เป็นเครื่องมือแบบ Interactive ที่ทรงพลังสำหรับการเขียนโค้ดและการวิเคราะห์ข้อมูล ชื่อ "Jupyter" มาจากการผสมผสานภาษาโปรแกรมหลักที่รองรับในยุคแรกคือ Julia, Python และ R

เอกสารของ Jupyter ที่เรียกว่า "โน้ตบุ๊ก" (Notebook) ประกอบด้วย เซลล์ (Cells) ซึ่งสามารถเป็นได้ทั้งโค้ดที่รันได้ (Code Cell) หรือข้อความที่จัดรูปแบบได้ (Markdown Cell) ทำให้สามารถผสมผสานโค้ด, ผลลัพธ์, กราฟ, และคำอธิบายไว้ในเอกสารฉบับเดียวกันได้อย่างลงตัว



2.4.1 การติดตั้งและเปิดใช้งาน Jupyter Notebook

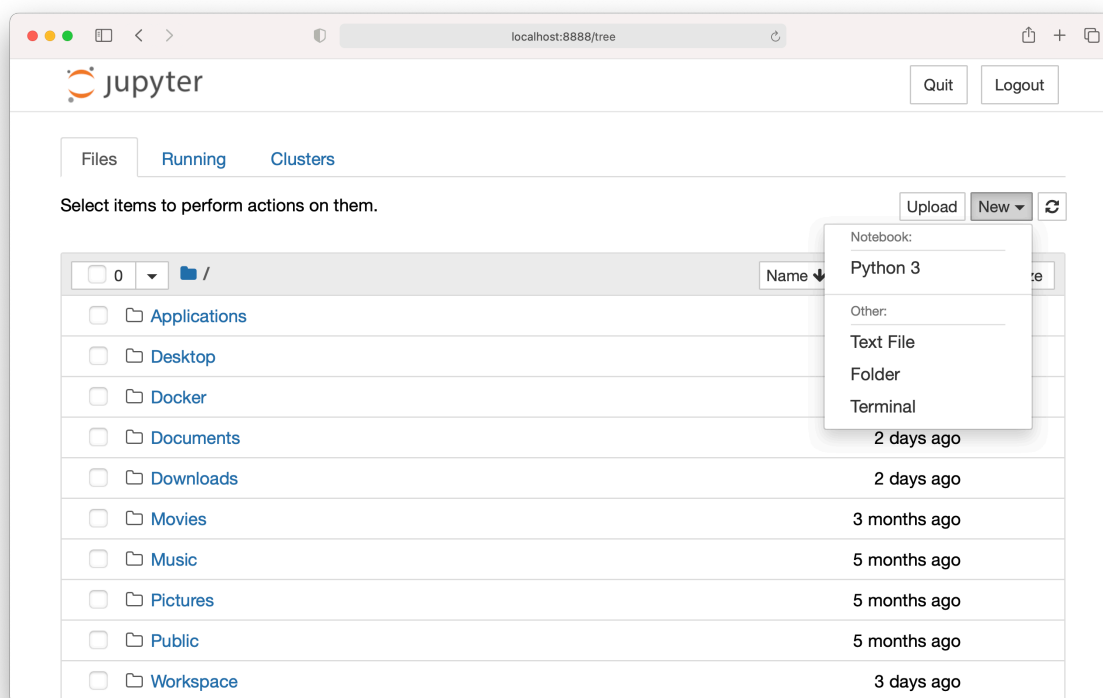
เราสามารถติดตั้ง Jupyter Notebook ได้อย่างง่ายดายผ่าน 'pip'

```
pip install notebook
```

หลังจากติดตั้งเสร็จสิ้น ให้เปิดใช้งานด้วยคำสั่ง

```
jupyter notebook
```

คำสั่งนี้จะเปิดหน้า Dashboard ในเว็บเบราว์เซอร์ของคุณ ซึ่งทำหน้าที่เป็นศูนย์กลางในการจัดการไฟล์และโน้ตบุ๊กต่างๆ



รูป 2.2: หน้า Dashboard ของ Jupyter Notebook (Jupyter Notebook Dashboard)

นอกจากนี้ ยังมีบริการบนคลาวด์ที่ให้ใช้งาน Jupyter Notebook ได้ฟรี เช่น Google Colaboratory⁵ ซึ่งเป็นตัวเลือกที่ยอดเยี่ยมสำหรับผู้ที่ต้องการเริ่มต้นอย่างรวดเร็วโดยไม่ต้องติดตั้งโปรแกรมบนเครื่อง

2.4.2 การใช้งานพื้นฐาน

- **การสร้างโน้ตบุ๊กใหม่:** จากหน้า Dashboard คลิกที่ **New** ► **Python 3 (ipykernel)**
- **เซลล์โค้ด (Code Cell):** เป็นพื้นที่สำหรับเขียนและรันโค้ดไพธอน คุณสามารถรันเซลล์ได้โดยการกด  + 
- **เซลล์มาร์คดาวน์ (Markdown Cell):** ใช้สำหรับเขียนข้อความ, หัวข้อ, รายการ, หรือใส่ลิงก์และรูปภาพ โดยเปลี่ยนประเภทเซลล์จากเมนู Dropdown บน Toolbar
- **การบันทึก:** โน้ตบุ๊กจะถูกบันทึกอัตโนมัติเป็นระยะ แต่สามารถบันทึกด้วยตนเองได้โดยกดที่ไอคอนรูปแผ่นดิสก์

⁵Google Colab: <https://colab.research.google.com>

2.5 การแสดงผลและการรับข้อมูล

2.5.1 การแสดงผลด้วยฟังก์ชัน print()

ฟังก์ชัน `print()` เป็นฟังก์ชันพื้นฐานสำหรับแสดงผลข้อมูลออกทางหน้าจอ

```
print(*objects, sep=' ', end='\n')
```

- `objects`: สิ่งที่ต้องการให้แสดงผล สามารถมีได้หลายค่า คั่นด้วยจุลภาค (,)
- `sep`: ตัวคั่นระหว่างแต่ละ object (ค่าเริ่มต้นคือเว้นวรรค)
- `end`: อักขระที่จะพิมพ์ปิดท้าย (ค่าเริ่มต้นคือขึ้นบรรทัดใหม่ \n)

ตัวอย่าง 2.1 การใช้ฟังก์ชัน print() รูปแบบต่างๆ

```
1 # แสดงข้อความและตัวเลข
2 print("Welcome to Python version", 3.13)
3
4 # ใช้พารามิเตอร์ sep
5 print("apple", "banana", "cherry", sep=", ")
6
7 # ใช้พารามิเตอร์ end
8 print("Loading", end="")
9 print("...", end="")
10 print(" Done!")
```

```
Welcome to Python version 3.13
apple, banana, cherry
Loading... Done!
```

OUTPUT

การจัดรูปแบบสตริงด้วย **f-string** ในไพทอนเวอร์ชันใหม่ๆ (3.6+) วิธีที่นิยมและสะดวกที่สุดในการผสมค่าจากตัวแปรลงในสตริงคือการใช้ f-string โดยนำหน้าสตริงด้วยตัวอักษร f และใส่ชื่อตัวแปรไว้ในวงเล็บปีกกา {}

ตัวอย่าง 2.2 การใช้ f-string ในการแสดงผล

```
1 name = "Alice"
2 age = 30
3 print(f"Hello, my name is {name} and I am {age} years old.")
```

```
Hello, my name is Alice and I am 30 years old.
```

OUTPUT

2.5.2 การรับข้อมูลด้วยฟังก์ชัน input()

ฟังก์ชัน `input()` ใช้สำหรับรับข้อมูลจากผู้ใช้ผ่านทางคีย์บอร์ด โดยข้อมูลที่รับเข้ามาทั้งหมดจะอยู่ในรูปแบบสตริง (String) เสมอ

ตัวอย่าง 2.3 การใช้งานฟังก์ชัน `input()`

```
1 name = input('กรุณาป้อนชื่อของคุณ: ')
2 print(f'สวัสดีครับ/ค่ะ, คุณ {name}!')
```

กรุณาป้อนชื่อของคุณ: สมชาย
สวัสดีครับ/ค่ะ, คุณ สมชาย!

OUTPUT

2.6 การเขียนคำอธิบายโปรแกรม (Comments)

การเขียนคำอธิบายหรือคอมเมนต์เป็นส่วนสำคัญในการทำให้โค้ดของเราอ่านเข้าใจง่าย ทั้งสำหรับตัวเราเองในอนาคตและสำหรับผู้อื่นที่มาอ่านโค้ด

1. **คอมเมนต์บรรทัดเดียว (Single-line comment):** ใช้เครื่องหมาย `#` นำหน้าข้อความ
2. **คอมเมนต์หลายบรรทัด (Multi-line comment):** ใช้เครื่องหมาย Triple Quotes `'''...'''` หรือ `"""..."""` ครอบข้อความ

ตัวอย่าง 2.4 ตัวอย่างการเขียนคอมเมนต์

```
1 # This line calculates the area of a rectangle.
2 width = 10
3 height = 20
4 area = width * height # width * height
5
6 """
7 This is a multi-line comment block.
8 It can span across multiple lines and is often used
9 for explaining complex functions or sections of code.
10 """
11 print(f"The area is: {area}")
```

The area is: 200

OUTPUT

สรุปท้ายบท

ในบทนี้ เราได้ปูพื้นฐานที่จำเป็นสำหรับการเริ่มต้นเขียนโปรแกรมด้วยภาษาไพธอน ตั้งแต่การติดตั้งตัวแปลภาษา, ความสำคัญของสภาพแวดล้อมเสมือน, การรันสคริปต์เบื้องต้น, ไปจนถึงการใช้งานเครื่องมือยอดนิยมอย่าง Jupyter Notebook นอกจากนี้เรายังได้เรียนรู้ฟังก์ชันพื้นฐานสำหรับการรับและแสดงผลข้อมูลอย่าง `input()` และ `print()` รวมถึงการจัดรูปแบบข้อความด้วย f-string และการเขียนคอมเมนต์เพื่อให้โค้ดอ่านง่ายขึ้น ความรู้ทั้งหมดนี้เป็นรากฐานสำคัญที่จะทำให้เราสามารถพัฒนาโปรแกรมที่ซับซ้อนยิ่งขึ้นในบทต่อไป

แบบฝึกหัด

1. โปรแกรมแก้ไขโค้ด (Text Editor) และสภาพแวดล้อมการพัฒนาแบบผสมผสาน (IDE) แตกต่างกันอย่างไร? จงยกตัวอย่างเครื่องมือของแต่ละประเภทอย่างน้อย 1 ชนิด
2. สภาพแวดล้อมเสมือน (Virtual Environment) ในการพัฒนาโปรแกรมภาษาไพธอนมีประโยชน์หลักอย่างไร?
3. จงเขียนโปรแกรมรับ "ชื่อ" และ "อายุ" ของผู้ใช้ จากนั้นให้แสดงผลเป็นประโยคแนะนำตัวโดยใช้ f-string เช่น "สวัสดี, ผมชื่อสมชาย ปัจจุบันอายุ 25 ปี"
4. ฟังก์ชัน `print()` มีพารามิเตอร์ `sep` และ `end` ไว้เพื่อจุดประสงค์ใด? จงเขียนโค้ดหนึ่งบรรทัดเพื่อแสดงคำว่า "Python", "is", "fun" โดยให้คั่นแต่ละคำด้วยเครื่องหมาย " - " และจบประโยคด้วยเครื่องหมาย "!" โดยไม่มีการขึ้นบรรทัดใหม่
5. จงเขียนโปรแกรมรับค่าตัวเลข 2 จำนวนจากผู้ใช้ แล้วแสดงผลบวกของเลขทั้งสอง (คำใบ้: ข้อมูลที่รับจาก `input()` เป็นสตริงเสมอ คุณอาจต้องค้นหาวิธีแปลงสตริงเป็นตัวเลข)
6. ใน Jupyter Notebook, เซลล์ประเภท "Code" และ "Markdown" ใช้ทำอะไรและแตกต่างกันอย่างไร?
7. จงอธิบายความสำคัญของการเขียนคำอธิบายโปรแกรม (Comment) และยกตัวอย่างการคอมเมนต์ทั้งแบบบรรทัดเดียวและหลายบรรทัด
8. จงเขียนโปรแกรมเพื่อแสดงข้อมูลส่วนตัว (ชื่อ, คณะ, มหาวิทยาลัย) โดยจัดรูปแบบการแสดงผลให้อยู่ในรูปแบบของบัตรนักศึกษาอย่างง่าย
9. หากต้องการติดตั้งไลบรารีเพิ่มเติมในภาษาไพธอน เช่น `numpy` สำหรับการคำนวณทางวิทยาศาสตร์ เราควรใช้เครื่องมือใด และใช้คำสั่งพื้นฐานอะไรในการติดตั้ง?
10. จงเขียนโปรแกรมเพื่อคำนวณดัชนีมวลกาย (BMI) โดยรับค่าน้ำหนัก (กิโลกรัม) และส่วนสูง (เมตร) จากผู้ใช้ แล้วแสดงผลพร้อมข้อความที่เหมาะสม (สูตร: $BMI = \frac{\text{น้ำหนัก (kg)}}{\text{ส่วนสูง (m)}^2}$)

เฉลยแบบฝึกหัด

1. **ความแตกต่าง:** Text Editor เป็นโปรแกรมสำหรับแก้ไขข้อความทั่วไปที่อาจมีฟังก์ชันเสริมสำหรับเขียนโค้ด เช่น Syntax Highlighting แต่ IDE เป็นชุดเครื่องมือที่ครบวงจรซึ่งรวมเอา Text Editor, Debugger, Compiler/Interpreter และเครื่องมืออื่นๆ ไว้ด้วยกันเพื่อการพัฒนาโปรแกรมโดยเฉพาะ
ตัวอย่าง: Text Editor: Visual Studio Code, Sublime Text. IDE: PyCharm, Spyder.
2. **ประโยชน์ของ Virtual Environment:** ช่วยแยกโปรเจกต์และไลบรารีที่แต่ละโปรเจกต์ต้องใช้ออกจากกัน ทำให้สามารถจัดการเวอร์ชันของไลบรารีที่แตกต่างกันในแต่ละโปรเจกต์ได้โดยไม่เกิดความขัดแย้ง และทำให้สภาพแวดล้อมการพัฒนาสะอาดและเป็นระเบียบ
3. **โปรแกรมแนะนำตัว:**

```

1 name = input("กรุณาป้อนชื่อของคุณ: ")
2 age = input("กรุณาป้อนอายุของคุณ: ")
3 print(f"สวัสดี, ผมชื่อ{name} ปัจจุบันอายุ {age} ปี")

```

4. พารามิเตอร์ **sep** และ **end**: **sep** ใช้กำหนดตัวคั่นระหว่างออบเจกต์ที่ส่งให้ **print()** ส่วน **end** ใช้กำหนดอักขระที่จะพิมพ์ต่อท้ายสุดแทนการขึ้นบรรทัดใหม่

```
1 print("Python", "is", "fun", sep=" - ", end="!")
```

5. โปรแกรมบวกเลข: เราต้องแปลงสตริงที่รับเข้ามาให้เป็นตัวเลขก่อน (เช่น float) โดยใช้ฟังก์ชัน **float()**

```
1 num1_str = input("ป้อนตัวเลขที่ 1: ")
2 num2_str = input("ป้อนตัวเลขที่ 2: ")
3 result = float(num1_str) + float(num2_str)
4 print(f"ผลบวกของ {num1_str} และ {num2_str} คือ {result}")
```

6. เซลล์ใน Jupyter: เซลล์ "Code" ใช้สำหรับเขียนและรันโค้ดภาษาโปรแกรม เช่น Python และจะแสดงผลลัพธ์ของโค้ดนั้นๆ ส่วนเซลล์ "Markdown" ใช้สำหรับเขียนข้อความ, จัดรูปแบบเอกสาร, ใส่หัวข้อ, รูปภาพ หรือสมการ เพื่อสร้างคำอธิบายประกอบโค้ด

7. ความสำคัญของ Comment: ช่วยอธิบายการทำงานของโค้ดที่ซับซ้อน, เตือนความจำเกี่ยวกับตรรกะของโปรแกรม, และช่วยให้ผู้ร่วมงานเข้าใจโค้ดได้ง่ายขึ้น ทำให้การบำรุงรักษาและแก้ไขโปรแกรมในอนาคตทำได้สะดวก

```
1 # นี่คือนิยามเมธอดบวกรหัสเดียว
2 x = 10 # กำหนดค่าเริ่มต้นให้กับ x
3
4
5 # นี่คือนิยามเมธอด
6 # แบบหลายบรรทัด
7 # ใช้สำหรับอธิบายภาพรวม
8
9
```

8. โปรแกรมบันทึกนักศึกษา:

```
1 name = input("ชื่อ-นามสกุล: ")
2 faculty = input("คณะ: ")
3 university = input("มหาวิทยาลัย: ")
4
5 print("\n=====")
6 print(f"      {university.upper()}")
7 print("-----")
8 print(f"ชื่อ: {name}")
9 print(f"คณะ: {faculty}")
10 print("=====")
```

9. การติดตั้งไลบรารี: เราใช้เครื่องมือที่เรียกว่า 'pip' (Pip Installs Packages) ซึ่งเป็นผู้จัดการแพ็คเกจมาตรฐานของไพธอน คำสั่งพื้นฐานคือ:

```
pip install numpy
```

10. โปรแกรมคำนวณ BMI:

```
1 weight_kg = float(input("ป้อนน้ำหนัก (กิโลกรัม): "))
2 height_m = float(input("ป้อนส่วนสูง (เมตร): "))
3
4 bmi = weight_kg / (height_m ** 2)
5
6 print(f"คุณมีดัชนีมวลกาย (BMI) เท่ากับ: {bmi:.2f}")
```

